27 LIFECYCLE EVENTS — HOOKS FOR APP STARTUP/SHUTDOWN (NOT PER-REQUEST)

Every other theory file about middleware/guards/pipes/filters (theory/25, 17, 09, 24) describes the PER-REQUEST lifecycle — what happens to ONE incoming HTTP call. Lifecycle events are a completely different axis: hooks that fire once as the whole Nest APPLICATION starts up and shuts down, regardless of how many requests ever arrive.

PER-REQUEST (fires every call)	APPLICATION LIFECYCLE (fires once)
Middleware → Guards → Pipes →	Module init hooks (app starting)
Handler → Response	| app.listen() starts accepting requests | ... many requests handled ... | Module destroy hooks (app stopping)

THE HOOKS, IN THE ORDER THEY ACTUALLY FIRE

Nest calls these on any provider/controller that implements the matching interface. All are optional — implement only the ones you need.

1. OnModuleInit Fires once each module's dependencies have been resolved (DI is done, so `this.someInjectedService` is safe to use). Runs BEFORE the app starts listening for requests.	onModuleInit()
2. OnApplicationBootstrap Fires after ALL modules have finished onModuleInit — i.e. once the ENTIRE dependency graph is fully initialized, still before the app accepts requests. Use this instead of OnModuleInit when a hook needs OTHER modules to already be fully ready, not just its own.	onApplicationBootstrap()

```
... app.listen() ...  <-- requests now start flowing in
```

3. OnModuleDestroy Fires when a shutdown signal is received (see "enabling shutdown hooks" below), before the app actually stops.	onModuleDestroy()

4. BeforeApplicationShutdown Fires after all onModuleDestroy hooks, still before the app shuts down. Receives the OS signal (e.g. 'SIGTERM') that triggered shutdown.	beforeApplicationShutdown(signal)
5. OnApplicationShutdown Fires last, as the final step before the process exits.	onApplicationShutdown(signal)

A REAL EXAMPLE IN THIS PROJECT'S SHAPE — TaskService SEEDING DATA

TaskService currently holds an in-memory array with no seed data step. If it needed one, OnModuleInit is exactly the right hook — DI has already run, so any injected repository/service is ready to use:

```
import { Injectable, OnModuleInit } from '@nestjs/common';

@Injectable()
export class TaskService implements OnModuleInit {
  private tasks: Task[] = [];

  onModuleInit() {
    console.log('TaskService ready — seeding initial tasks');
    this.tasks.push({ id: 1, title: 'Seed task', done: false });
  }
}
```

Why not just do this in the constructor? Because the constructor only guarantees THIS class's own dependencies exist — it does not guarantee they're safe to CALL yet (e.g. a database connection object might exist but not be connected). onModuleInit runs once Nest considers the module's initialization phase complete, which is the conventional place for "now it's safe to actually use my dependencies" logic.

CLEANUP ON SHUTDOWN — enableShutdownHooks

OnModuleDestroy/OnApplicationShutdown do NOT fire automatically on Ctrl+C — Nest needs to be told to listen for OS signals first:

```
// main.ts
async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.enableShutdownHooks(); // without this, destroy hooks never fire
  await app.listen(3000);
}
```

Typical use: closing a database connection pool cleanly instead of letting the process die mid-write.

```
@Injectable()
export class TaskService implements OnModuleDestroy {
  onModuleDestroy() {
    console.log('App shutting down — closing resources');
```

```
  }
}
```

HOW THIS RELATES TO MIDDLEWARE (theory/25)

It's easy to conflate these because both sound like "something that runs early." The distinction:

```
LoggeerMiddleware.use()   Runs on EVERY request, forever, for the
                          life of the app. Registered once via
                          configure() in app.module.ts, but its
                          CODE executes repeatedly.

onModuleInit()            Runs EXACTLY ONCE, when the app starts,
                          before any request is even possible.
```

If you wanted to log "server starting up" one time (vs. logging every request the way LoggeerMiddleware does), onModuleInit/onApplicationBootstrap in main.ts's bootstrap function (or on any provider) is the right tool, not middleware.

★ RULE OF THUMB

Reach for a lifecycle hook when the concern is about the APPLICATION's state (has DI finished, is a DB connected, should I clean up before exiting) — not about any individual HTTP request. onModuleInit is the common one (seed data, verify a connection, warm a cache); onModuleDestroy/enableShutdownHooks matter the moment the app owns any resource that needs a clean close (DB pools, open sockets, file handles). Everything else that "runs early" per REQUEST is middleware (theory/25), not a lifecycle hook.

See also: theory/25-middleware.txt, theory/07-dependency-injection.txt, theory/01-module.txt